

FSM Design for I2C Master Recovery Mechanism

¹Paradi Ramesh, ²A Venkateswara Rao, ³CH Subba Rao

¹PG Scholar, ²Associate Professor, ³Assistant Professor

^{1,2,3} Department of Electronics and Communication Engineering

^{1,2,3} Miracle Educational Society Group of Institutions, Bhogapuram-535216, AP, India

Abstract – FSM design for I2C master recovery mechanism is proposed in this thesis, this recovery mechanism has different states. The I2C or Inter-Integrated Circuit protocol is a serial communication protocol. I2C is a bidirectional 2-wire bus designed to enhance hardware efficiency and increase simplicity of the circuit. This protocol bolsters multiple masters (which is a limitation with SPI communication) and multiple slaves and also allows communication between faster and slower devices by a serial data bus (SDA) without data loss. The other line is Serial clock line (SCL) which transfers the data according to a synchronized clock which is a limitation for UART communication. These two pins communicate using particular commands like start, address, read/write, acknowledgement and stop commands. Other types of communication protocols like USB, RS-422, RS-485, CAN etc. require more pin connections and signals for communication. The postulates of I2C over UART, SPI, USB and other protocols indicates I2C's significance in its use as communication protocol. This thesis makes use of Verilog language in designing and implementing I2C bus on EDA playground (Aldec Riviera 2025.05), verified and simulated the FSM for I2C protocol recover mechanism.

Index Terms – I2C Interface, serial clock, serial data, bus recovery mechanism.

I. INTRODUCTION

The I2C protocol is widely used for communication between integrated circuits, and this project aims to provide a practical and functional Verilog implementation of the protocol. Analyzed waveform outputs to verify correct I2C timing and communication.

Design: Contains the Verilog HDL source code files for the I2C protocol implementation.

Block Diagram: Provides a visual representation of the design architecture.

Testbench: Includes the Verilog testbench files used to verify the functionality of the I2C implementation.

State Diagram: Presents a state diagram illustrating the behavior of the Finite State Machine (FSM) used in the I2C implementation.

Results: Displays the simulation results demonstrating the successful operation of the I2C protocol.

The I2C specification provides a standardized recovery mechanism known as "Bus Clear" or "Bus Recovery."

This protocol leverages the fact that while SDA (Serial Data) might be held low by a confused slave, the SCL (Serial Clock) line typically remains under master control.

The Bus Clear sequence operates on a fundamental principle: since I2C slaves only drive SDA low (never high), a series of clock pulses can eventually free any slave from its stuck state by allowing it to complete its interrupted transmission.

Most of the time a few extra clock cycles is enough to get the slave back in sync. The standard recovery function will do the following:

- Check if the bus is really stuck (SCL or SDA low)
If the bus is stuck, it will try to toggle the SCL line up to 9 times (to cover a full byte transfer)
- After each clock cycle, it will check if the SDA line is released (high)
If the SDA line is released, it will generate a STOP condition by pulling SDA low while SCL is high, and then release SDA.
If the bus is still stuck after 9 clock cycles, it will return an error.

II. RELATED WORK

Here's the waveform generated from I2C, showing transitions for the following signals over time:

- **clk:** Clock signal used to synchronize the I2C communication.
- **rst:** Reset signal, typically active-high to reset the state machine or registers.

- **scl**: Serial Clock Line for I2C — controlled by the master to coordinate data bits.
- **sda**: Serial Data Line for I2C — used for sending/receiving bits during communication.

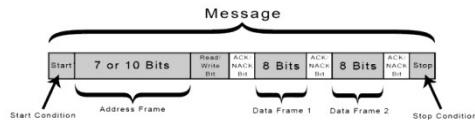


Figure 1: I2C protocol

A. Analyze key events[3]

- **START condition**: sda goes from high to low while scl is high.
- **STOP condition**: sda goes from low to high while scl is high.
- **Data bit transmission**: occurs while scl is toggling and sda is stable.

B. Core Recovery Algorithm[4]

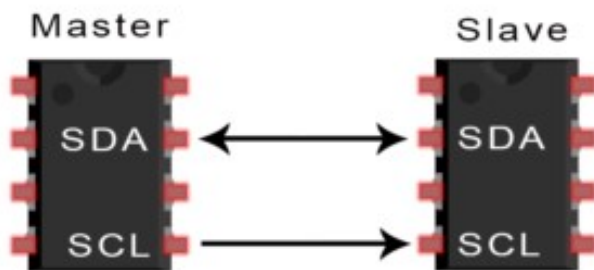


Figure 2: I2C design architecture

The bus clear sequence follows these essential steps:

- **Detection Phase**: Identify that SDA is stuck low while SCL is high
- **Clock Generation**: Generate up to 9 SCL pulses to flush slave state
- **STOP Condition**: Send proper I2C STOP to reset all slaves
- **Bus Validation**: Verify both SDA and SCL return to idle high state
- **Re-initialization**: Restore normal I2C controller operation

This approach mirrors biological healing processes—when sensors encounter errors, the system automatically initiates recovery protocols, much like how Hoomanely's biosense AI engine responds to anomalous health data patterns.

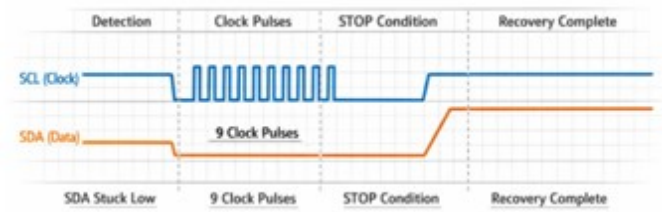


Figure 3: Timing diagram of I2C design

C. Finite State Machine for I2C[5]

This FSM of the bus represents the data transfer format explained above. The diagram for algorithm of the FSM is shown below.

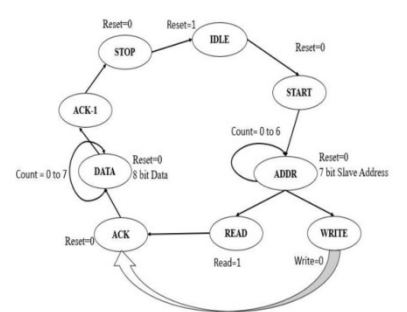


Figure 4: FSM of the bus data transfer format

Algorithm: 1

STATE_IDLE: In this state the I2C bus does not perform any operation and the bus is idle. Both the SDA and SCL lines remain high in this state. The reset bit remains high during this state and the bus moves to next state only when the reset bit becomes low shifting the bus to start state.

STATE_START: After reset becomes low this state is achieved and during this the FPGA master generates START condition for the initiation of data transfer shifting the bus to address frame. If reset becomes high in this state, the bus reverts back its changes and goes to STATE_IDLE. Also, in this state the count for the address frame is decided. As we were discussing the 7-bit address protocol the count declared here is 6.

STATE_ADDR: This state executes the address frame state and if reset becomes high the bus reverts back to idle. The count is decremented by 1 for each of the 7-bit address and until count is zero the bus stays in this state. After annulation of the count the bus is moved to 8th bit state which is R₀ bit. Is reset is

high during this state the bus returns to STATE_IDLE.

STATE_RW: After the address frame, this state decides the operation whether it is read or write depending on this bit. If this bit is high the operation is read otherwise it is write operation and this is acknowledged by the slave.

STATE_ACK: This is the first acknowledgment for address frame by the slave and it transfers the bus for initiation of data frame. During this state, the count for data state is decided and as the data is byte format the count is 7.

STATE_DATA: In this count is decremented by 1 for each bit of the data and loop continues until count is zero. After that, bus is transferred to second acknowledgement.

STATE_ACK_2: This is the 2nd acknowledgment done by the slave for the data frame state.

STATE_STOP: The master generates STOP condition and terminates the action of carry by the bus by making both SDA and SCL high again. If reset becomes high after this the bus shifts to STATE_IDLE. If the reset stays zero, the bus makes the slave ready for another byte transfer by shifting to STATE_START

III. PROPOSED METHOD

This project involves writing synthesizable RTL code (Verilog) for each component,. The development process includes:

FSM is designed in Verilog for I2C master recovery mechanism at different states. The recovery mechanism is done when the data through the bus is corrupted. Different phases are used to recover the data when transferring through bus.

IV. RESULTS AND DISCUSSIONS

RTL level verification is completed and results are shown below:

Alternates between 0 and 1 consistently — the heartbeat of communication. Data bits are usually sampled on rising or falling edges.

rst: Starts high (1) then drops low (0), indicating the system is coming out of reset.

scl: Remains low initially (possibly while waiting for data), then toggles to simulate bit-level timing.

sda: Changes state when scl is low and typically held steady when scl is high

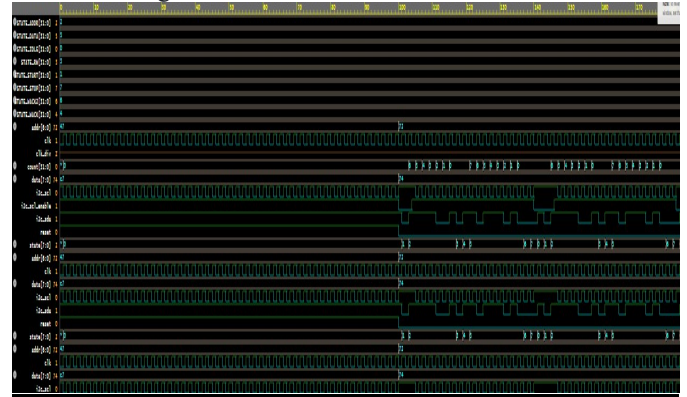


Figure 5: Simulated output waveform

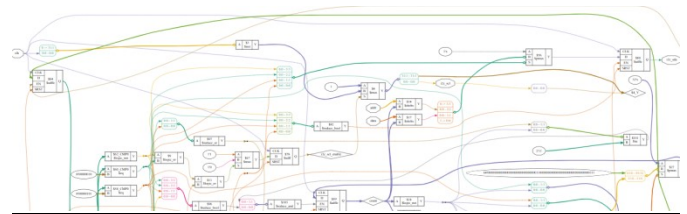


Figure 6: Synthesized output

V. CONCLUSION

I2C recovery mechanism is implemented and verified using FSM on data bus in Verilog with different flag register li ACK/NACK and some message bits after each data transfer.

REFERENCES

- [1] Randall Hyde No Starch Press, 11 Oct 2022 - Computers - 440 pages an extensive practical guide to connecting real-world devices to microcontrollers with the popular I2C bus.
- [2] Serial Port Complete [by Jan Axelson]: A classic engineering hardware reference providing code examples, bus layouts, and voltage level translation strategies for SPI and parallel interfaces.
- [3] Making Embedded Systems by Elecia White: Explains how to program synchronous serial communication directly from a software engineer's

Funding Declaration

The authors declare that no funds, grants, or other forms of financial support were received from any organization or institution for the conduct of this research.